# Automações Escaláveis em Python

Automações escaláveis com Python, Playwright, Asyncio e Docker



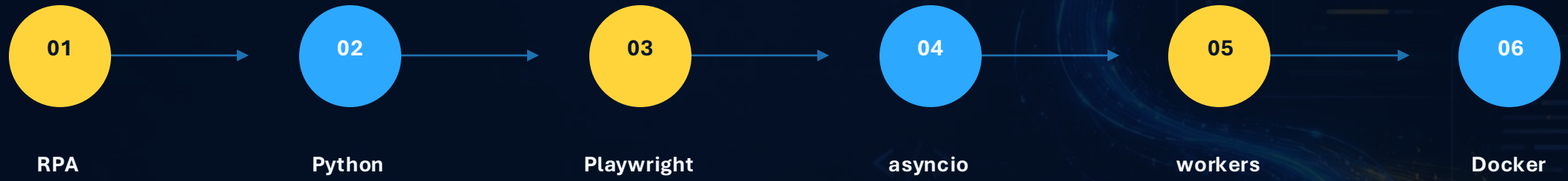
# Quem sou eu?





# A jornada de hoje

A apresentação é uma história: do robô simples ao sistema escalável.



# O destino final

No fim, a automação roda em ambiente reproduzível e escalável.

- Automação web com Playwright
- Processamento concorrente com asyncio
- Workers com contextos isolados
- Containerização com Docker
- Projeto clonado do GitHub



PARTE 1

# Automação como negócio

RPA, ROI, squads, governança e hiperautomação.



# O que é RPA

Automação da execução de tarefas repetitivas e baseadas em regras.

- Interage com sistemas como uma pessoa ou via integração.
- Resolve gargalos operacionais repetitivos.
- Funciona melhor quando processo, dados e regras estão claros.
- O desafio real começa quando vai para produção.



# RPA não é só clicar em tela

Automação madura conecta sistemas, dados e decisões.



# Onde está o retorno financeiro?

RPA precisa conversar com custo, risco e capacidade operacional.

## Manual

- horas/mês
- erros humanos
- retrabalho

## Automação

- execução assistida
- logs e auditoria
- padronização

## Negócio

- escala
- payback
- disponibilidade

# Ecossistema Python para automação

Python não é “uma ferramenta de RPA”: é a base para combinar ferramentas.

- Playwright
- Selenium
- requests
- pandas
- openpyxl
- PyAutoGUI
- OCR
- APIs
- LLMs



# RPA + LLMs

RPA executa. IA interpreta.




- Classificação de e-mails
- Extração de dados
- Decisão assistida

# Selenium × Playwright

Não é uma briga. É entender encaixe arquitetural.

## Selenium

- Muito consolidado
- Ecossistema enorme
- WebDriver
- Ótimo para legado

## Playwright

- Auto-wait
- Codegen
- API async
- BrowserContext

Neste tutorial, Playwright ajuda a ensinar isolamento, concorrência e containers com menos atrito.

# Por que Playwright aqui?

Porque ele combina com a história técnica que vamos construir.

- API assíncrona natural para asyncio.
- BrowserContexts leves e isolados.
- Auto-wait reduz código frágil.
- Codegen acelera o mapeamento inicial.
- Boa base para rodar em Docker.

# Como funciona um squad de automação

Automação é produto vivo, não script solto em uma pasta.




A conversa muda quando o robô entra em produção: versão, logs, credenciais, falhas e responsáveis.

# Python como plataforma: troca justa

Ganha liberdade, mas assume responsabilidades que plataformas RPA escondem.

## Ganha

- liberdade
- baixo lock-in
- ecossistema
- Performance (controle, otimização)

## Assume

- orquestração
- deploy
- observabilidade
- governança

# O problema das VMs

Escalar automação só com máquinas virtuais fica caro e difícil de governar.

- Cada automação pode exigir ambiente próprio.
- Browser, dependências e credenciais viram acoplamento.
- Custo e manutenção crescem rápido.
- Docker entra como resposta arquitetural mais adiante.



# Governança e sustentação

A automação não termina quando entra em produção.

- Quem executou? Quando? Com qual versão?
- Onde estão logs, prints e evidências?
- O que acontece se falhar na metade?
- Como fazer rollback e atualizar com segurança?
- Quem dá manutenção quando o site muda?

# Profissionais também são gargalo

Não basta saber Python. O perfil é multidisciplinar.

- Processos e regra de negócio.
- Automação web e integrações.
- Concorrência, filas e falhas.
- Infraestrutura, deploy e observabilidade.
- Comunicação com áreas de negócio.



# De RPA para hiperautomação

A tendência é automatizar processos, não telas.




Se existe API, use API. Se não existe, automatize interface. Se exige interpretação, combine IA.

# Perguntas?

**Antes da parte mão na massa**

# Intervalo

**20 minutos • água • café • banheiro**

PARTE 2

# Mão na massa

Agora vamos construir o caminho até a reference.

## Clone o repositório

Ponto de partida da prática: a reference é o destino; a aula é o caminho.

```
github.com/mmarques11/python-brasil-2026

# depois do clone ou faça uma fork
uv sync
uv run playwright install
```

# Repositório do tutorial

A reference é o destino; a pasta da aula é o caminho.

```
git clone https://github.com/mmarques11/python-brasil-2026
cd python-brasil-2026
```

**aula-python-brasil/**

- vamos construir juntos

**reference/**

- solução final funcionando

# Preparando o ambiente

Poucos comandos para chegar no primeiro robô rodando.

```
cd aula-python-brasil
uv sync
uv run playwright install chromium
```

- pyproject.toml declara dependências.
- uv.lock trava versões.
- .venv é criada automaticamente.
- uv run executa dentro do ambiente.

# uv × pip

Sem guerra religiosa: só o suficiente para o tutorial.

## pip

- instala pacotes
- fluxo tradicional
- requirements.txt

## uv

- ambiente + dependências
- lockfile
- execução do projeto

# Primeiro: automação síncrona

Antes de otimizar, precisamos ter uma automação funcionando.

```
with sync_playwright() as playwright:
    browser = playwright.chromium.launch(headless=False)
    page = browser.new_page()
    page.goto("https://rpachallenge.com/")
    page.get_by_role("button", name="Start").click()
```

- Abrir o site
- Mapear campos
- Preencher formulário
- Clicar em Submit
- Medir tempo total

# Temporizador: criando o baseline

Sem medir, otimização vira opinião.

```
inicio = datetime.now()
temporizador = perf_counter()

try:
    executar_processamento()
finally:
    duracao = perf_counter() - temporizador
    print(f"Tempo total: {duracao:.2f}s")
```

Primeiro medimos o fluxo síncrono. Depois criamos um motivo real para falar de concorrência.

# E se fossem 2.000 registros?

Agora o problema pede arquitetura

# Concorrência × paralelismo

Para automação web, muito tempo é espera de I/O.

## Concorrência

- tarefas alternam execução
- uma pausa libera outra
- ideal para I/O

## Paralelismo

- execução simultânea real
- usa múltiplos núcleos/processos
- bom para CPU-bound

# O que é asyncio

Uma ferramenta do Python para concorrência cooperativa baseada em I/O.

- Gerencia coroutines em um event loop.
- Permite avançar outras tarefas durante esperas.
- Funciona muito bem com rede, arquivos, browser e APIs.
- Não transforma automaticamente tudo em paralelo.



# Coroutine

async def não executa imediatamente: cria algo que pode pausar e retomar.

```
async def main() -> None:
    await executar_processamento()

coroutine = main()
```

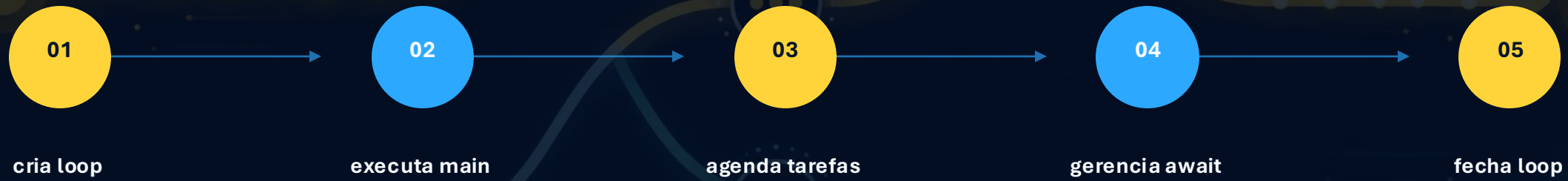
> Uma coroutine preserva estado local quando para em um await.

- pode suspender
- pode retomar
- precisa ser executada pelo event loop

# Event loop

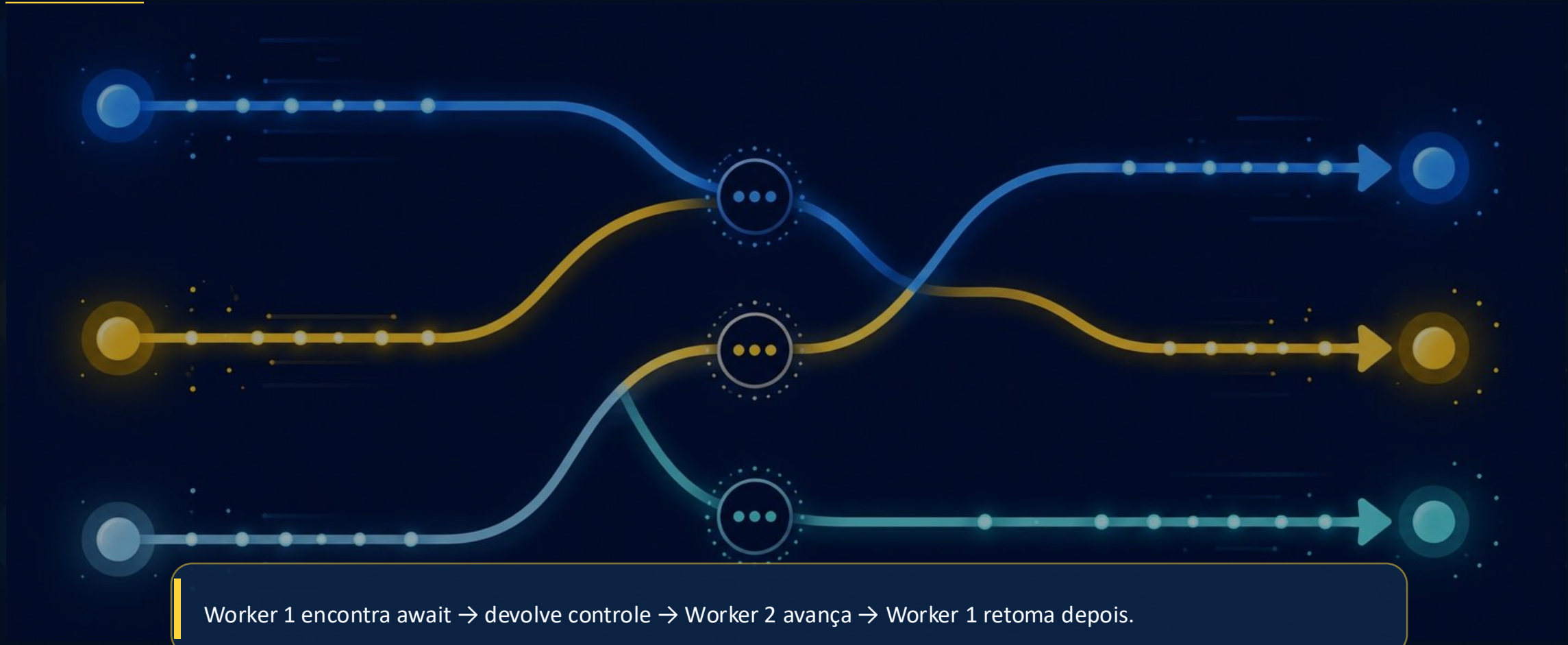
O coordenador que decide qual coroutine avança agora.

```
if __name__ == "__main__":
    asyncio.run(main())
```



# O await é onde a mágica aparece

Quando uma operação precisa esperar, outra coroutine pode avançar.



Worker 1 encontra await → devolve controle → Worker 2 avança → Worker 1 retoma depois.

# Context Manager

Gerencia entrada e saída de recursos sem esquecer limpeza.

```
with sync_playwright() as playwright:
    ...

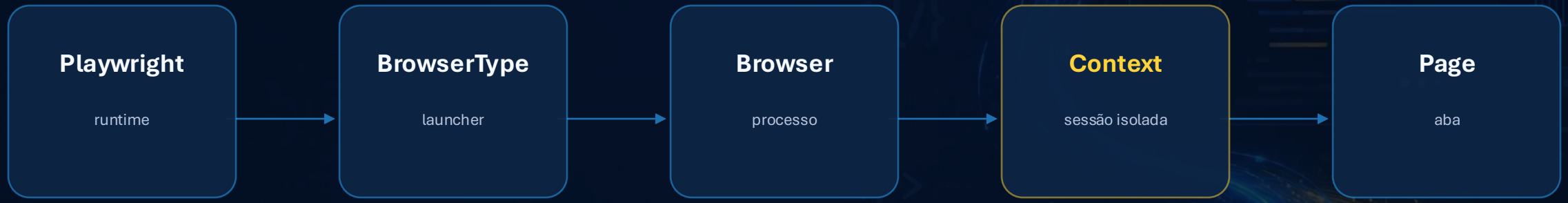
async with async_playwright() as playwright:
    ...
```

- Útil para browser, arquivos, conexões e sessões.
- async with permite limpeza assíncrona.



# Arquitetura Playwright

Quem é quem antes de criar workers.




```
browser = await playwright.chromium.launch()
context = await browser.new_context()
page = await context.new_page()
```

# BrowserContext: isolamento leve

Cada worker ganha uma sessão separada dentro do mesmo browser.

- Cookies separados.
- LocalStorage separado.
- Credenciais e sessão independentes.
- Mais leve do que abrir vários browsers.
- Perfeito para dividir trabalho em lotes.

# Criando workers

Cada worker recebe um lote e trabalha com seu próprio BrowserContext.

- Worker 1 → registros 1–3
- Worker 2 → registros 4–6
- Worker 3 → registros 7–10

No RPA Challenge, o objetivo aqui é demonstrar carga distribuída, não competir pelo score oficial.



# asyncio.gather

Executa várias coroutines concorrentemente e espera todas terminarem.

```
await asyncio.gather(
    *(
        executar_worker(driver, worker_id, lote)
        for worker_id, lote in enumerate(lotes, start=1)
    )
)
```

- ótimo quando já sabemos o conjunto de tarefas
- retorna quando todas terminam
- se uma falha, precisamos pensar em tratamento

# create_task

Agenda uma coroutine para rodar quando o event loop puder.

```
task = asyncio.create_task(worker())

print("faço outra coisa")

await task
```

- cria uma Task explicitamente
- permite continuar o fluxo
- não esqueça de dar await ou controlar a task

Bom momento para demonstrar prints e deixar o público “ver” o event loop alternando.

# Deixe o event loop visível

Debugger tradicional não mostra bem a troca entre coroutines. Logs mostram.

```
def log(msg: str) -> None:
    task = asyncio.current_task()
    nome = task.get_name()
    print(f"{nome}: {msg}")
```

```
0.01s | worker-1 | antes do await
0.02s | worker-2 | antes do await
0.03s | worker-3 | antes do await
2.01s | worker-1 | retomou
```

A explicação fica muito mais concreta quando a saída mostra quem pausou e quem retomou.

# Resultado: síncrono versus concorrente

A comparação fecha o ciclo: primeiro medimos, depois otimizamos.

## Síncrono

- um fluxo por vez
- esperas bloqueiam avanço

## Concorrente

- três workers
- esperas liberam o loop

Aqui você mostra os tempos reais da execução durante o tutorial.

PARTE 3

# Docker e escala

A automação funcionando vira uma unidade reproduzível.



# Docker entra em cena

Container é uma forma de empacotar aplicação, dependências e ambiente.

- Mesmo projeto rodando em máquinas diferentes.
- Menos “na minha máquina funciona”.
- Facilita deploy e repetibilidade.
- Base para escalar horizontalmente.



# Containerizando Playwright

O browser também faz parte do ambiente da automação.

```
FROM mcr.microsoft.com/playwright/python

WORKDIR /app
COPY . .
RUN uv sync --frozen
CMD ["uv", "run", "python", "main.py"]
```



- Python
- dependências do projeto
- browsers do Playwright
- comando de execução

Dockerfile

# Escalabilidade horizontal

Agora não pensamos só em mais coroutines, mas em mais unidades de execução.



1 container → 3 containers → 10 containers. A arquitetura começa a crescer como sistema.

# Workers + containers

Dois níveis diferentes de escala.

## Dentro do container

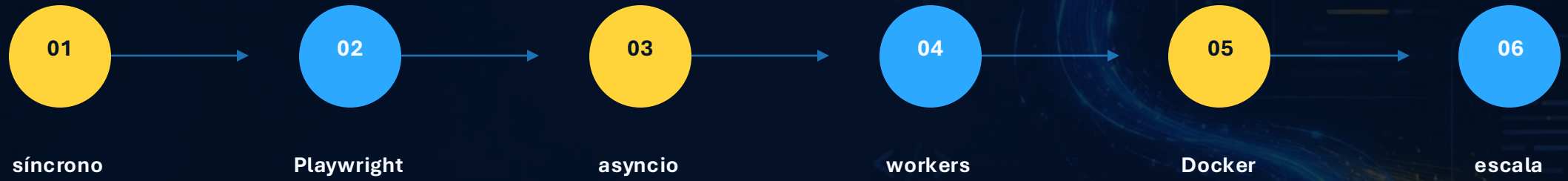
- asyncio
- 3 workers
- BrowserContexts

## Entre containers

- réplicas
- isolamento por processo
- escala horizontal

# O que construímos

Não foi apenas um robô: foi uma linha de evolução arquitetural.




Essa é a ponte entre automação de tarefa e engenharia de automação.

## A grande mensagem

# Automação escalável não é só fazer o robô executar mais rápido.

É construir uma arquitetura que consiga crescer, falhar, ser observada e ser sustentada.

# Obrigado!

Obrigado pela participação e pela troca de conhecimento.

**Perguntas?**

**Mateus Marques**

Python Brasil 2026 • Automações Escaláveis

Instagram: @mateusmarques04

LinkedIn: mateus-marques-42500b248